

EET 4501

***Prompt Engineering
Techniques***

Spring 2026

Manoj Kumal, PhD student

Dept. of Applied Computing

Michigan Tech



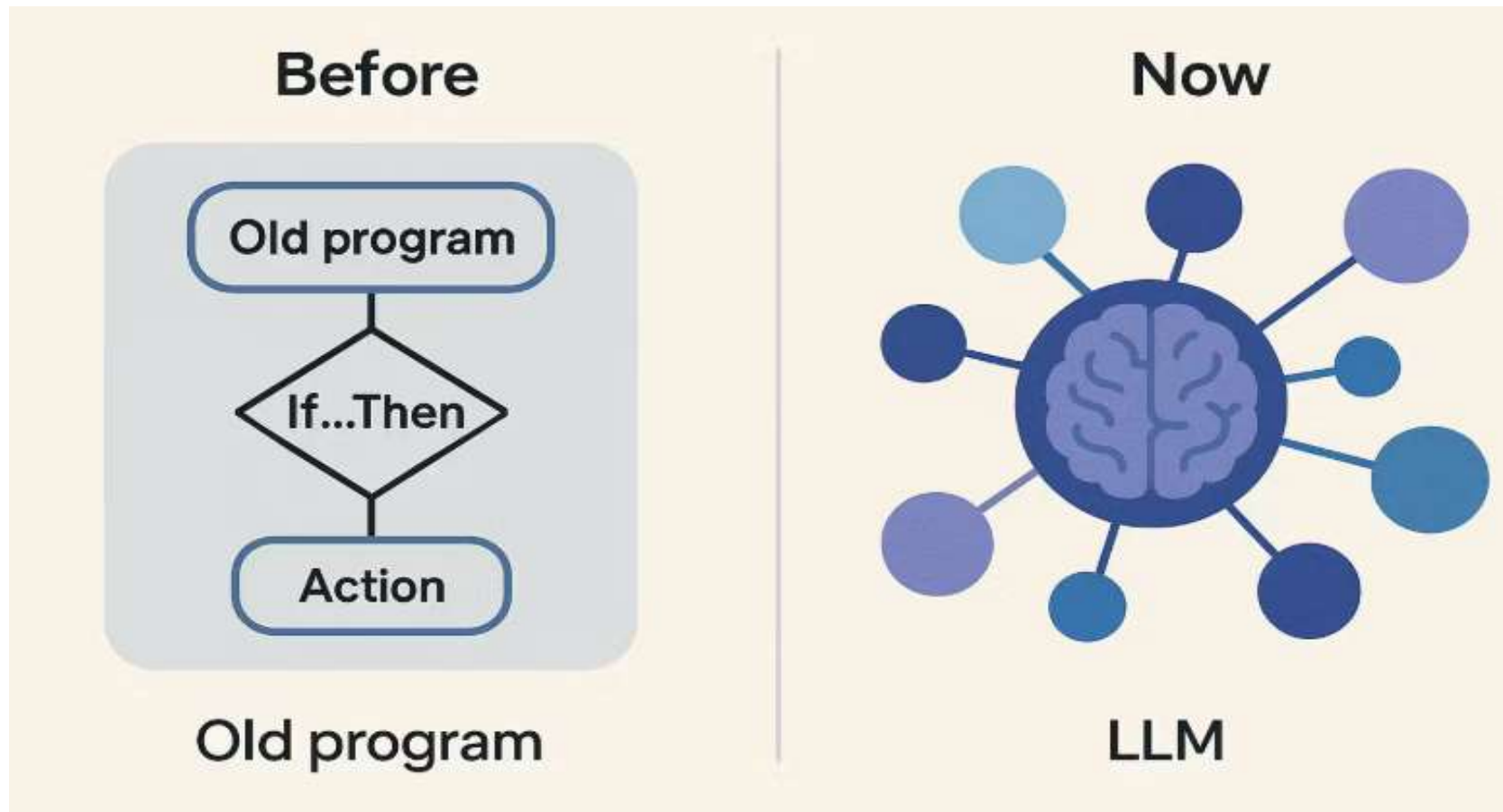
Agenda

- ❑ LLM
- ❑ **Why Prompting**
- ❑ **Prompt**
- ❑ **Prompting Techniques**



Large Language Model(LLM)

How AI understand text?



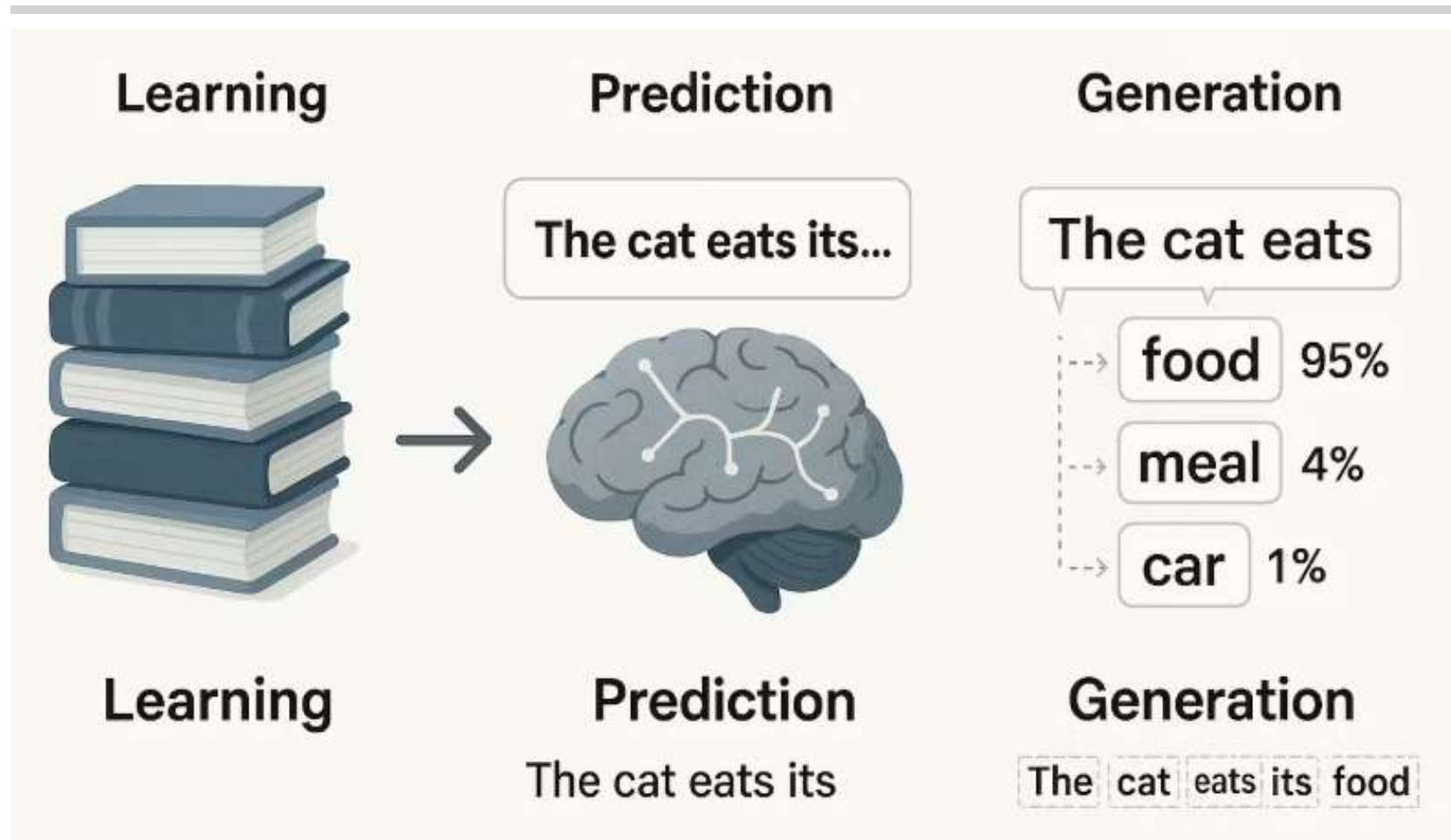
source: <https://www.swiftask.ai/blog/what-is-an-llm>

What is exactly LLM?



source: <https://imgflip.com/i/82cixh>

How a large language model works?



source: <https://www.swiftask.ai/blog/what-is-an-llm>



Why Prompting Matters

Simple Prompts



Why Prompting Matters?

- It determines the quality of the AI's output.
 - It saves time and effort by reducing the need for repeated questions.
 - It reduces incorrect or irrelevant answers.
 - It helps the AI better understand and match the user's intent.
 - It ensures the output follows the desired format (bullets, code, summary, json etc).
-



Prompt

What is prompt?

- A prompt is the input or instruction given to an AI or LLM.
 - It tells the AI what task to perform.
 - A prompt can be a question, command, or description.
 - The quality of the prompt affects the quality of the output.
 - Clear and specific prompts produce better and more accurate responses.
 - A prompt acts as a guide for the AI's response
-

BAD PROMPT Vs GOOD PROMPT

BAD PROMPT



“Write about marketing.”

- Too vague
- No context or direction
- Output will be generic and uninspiring

GOOD PROMPT



“Write a LinkedIn post about why storytelling in marketing works. Keep the tone professional yet engaging, add 3 bullet points, and end with a motivational one-liner.”

- Clear instructions
- Defined audience (LinkedIn readers)
- Expected format is specified
- Output becomes sharp, relevant, and engaging

source: <https://good-prompt-vs-bad-prompt-why-it-matters>

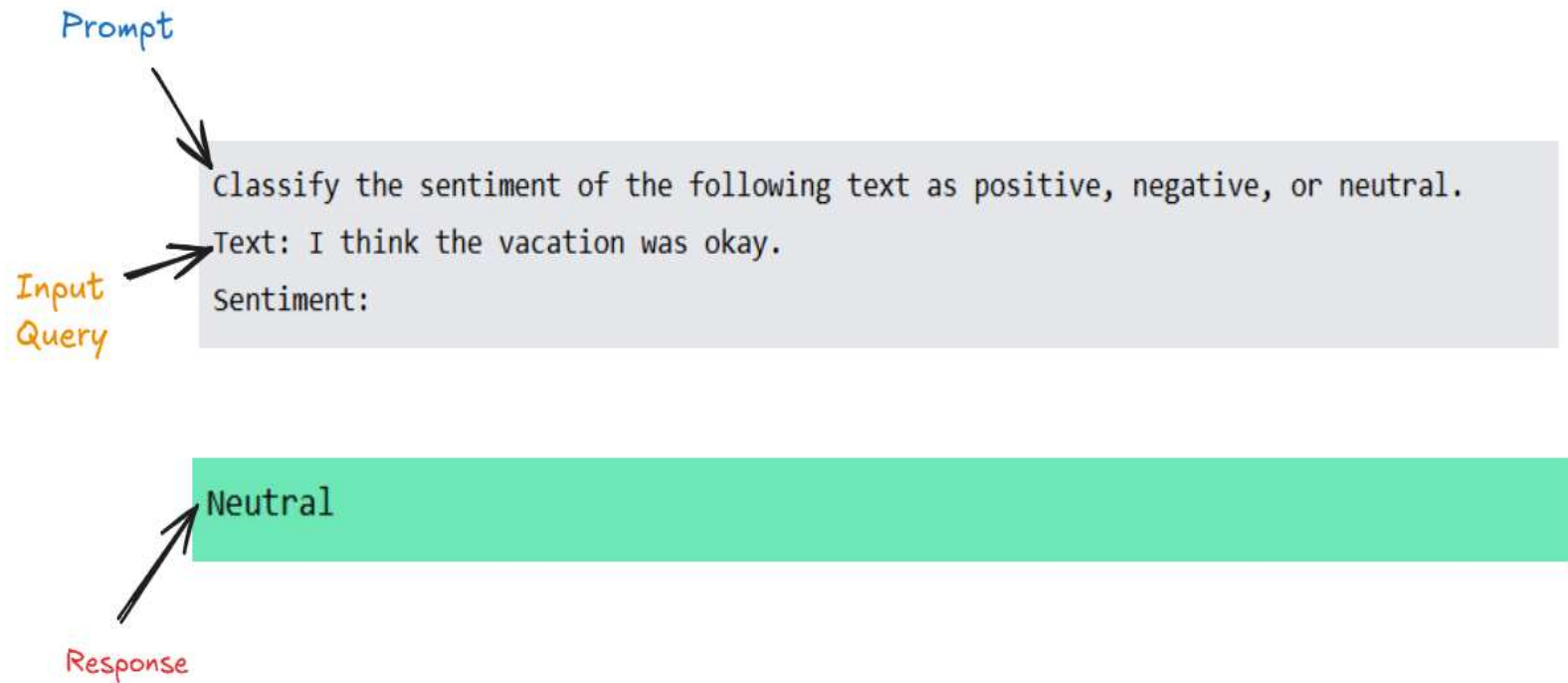


Prompting Techniques

Zero-Shot Prompting

- The AI is given only a task, with no examples.
 - The model relies on its pre-trained knowledge.
 - Fast and simple to use.
 - Works well for common or straightforward tasks.
-

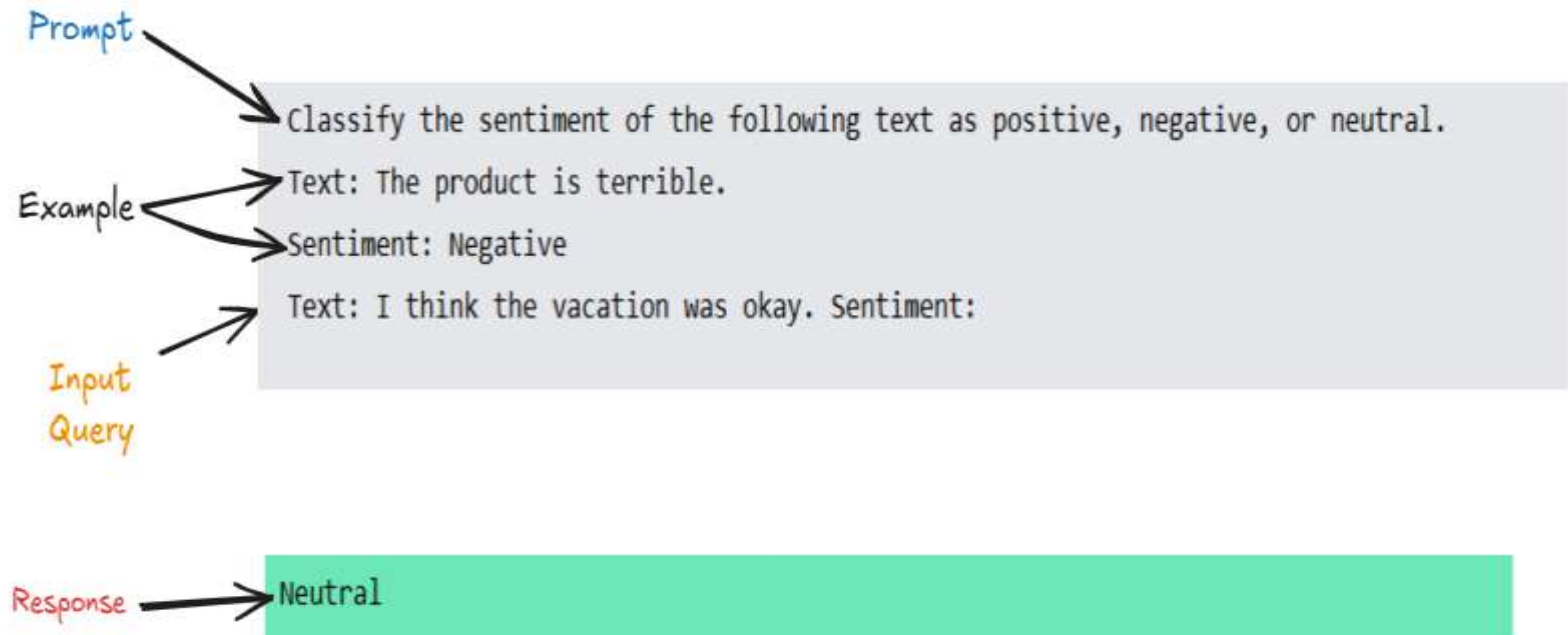
Zero-Shot Prompting Example



Few-Shot Prompting

- The AI is given a few examples along with the task.
 - Examples show the expected style, format, or logic.
 - Produces more accurate and consistent results.
 - Useful for complex or structured tasks.
-

Few-Shot Prompting Example



Chain of Thought(CoT) Prompting

- It encourages the AI to think step by step before giving the final answer.
 - It improves performance on reasoning and problem-solving tasks.
 - It helps the AI handle complex or multi-step questions.
 - It makes the reasoning process more transparent and understandable.
 - It reduces errors by guiding the model through logical steps.
-

Chain of Thought(CoT) Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

source: <https://arxiv.org/abs/2201.11903>

Chain of Thought(CoT) Prompting Example

The cafeteria had 23 apples. They used 20 apples to make lunch. They bought 6 more apples. How many apples do they have now?

Step 1: Identify the starting quantity

Initial apples = 23



Step 2: Account for apples used

Apples used = 20

Remaining apples = $23 - 20 = 3$



Step 3: Account for apples added

Apples bought = 6

Final total = $3 + 6 = 9$



The cafeteria has 9 apples.

Chain of Thought(CoT) Prompting

Example- code

```
from groq import Groq
prompt_cot = '''
Solve the problem step by step.

Henry had 11 dollars. He got 18 more and spent 10.
'''

client = Groq(
    api_key="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",
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": f"{prompt_cot}",
        }
    ],
    model="groq/compound-mini",
)

print(chat_completion.choices[0].message.content)
```

Chain of Thought(CoT) Prompting

Example- output

****Step-by-step solution****

1. ****Start with what Henry already has****

Henry initially has \\$11.

2. ****Add the money he receives****

He gets an additional \\$18.

\[

$$11 + 18 = 29$$

\]

After receiving the money, Henry has \$29.

3. ****Subtract the amount he spends****

He spends \$10.

\[

$$29 - 10 = 19$$

\]

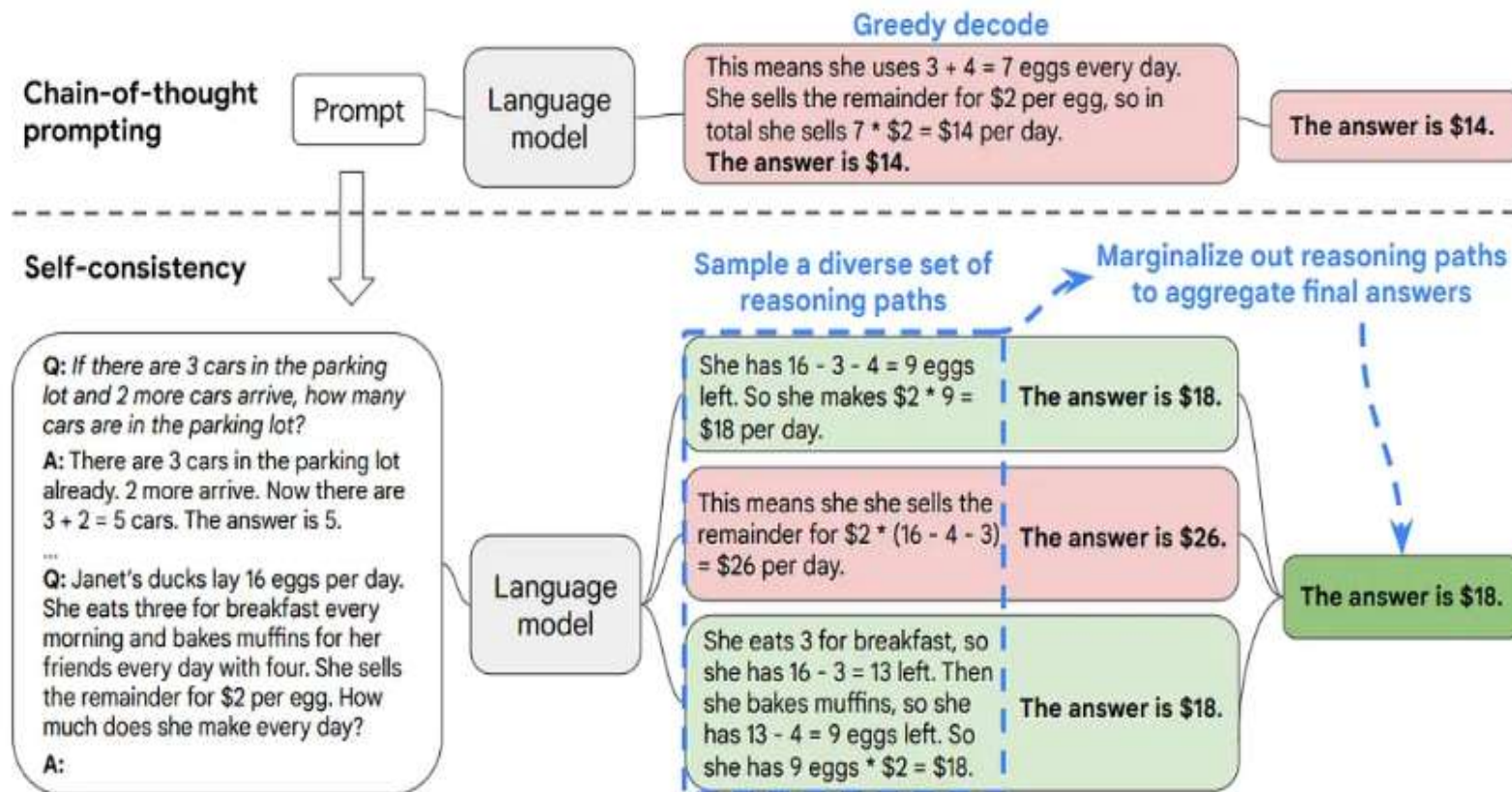
4. ****Result****

After getting the extra money and spending \$10, Henry ends up with ****\$19****.

Self Consistency

- The model generates several different reasoning paths for the same question rather than just one.
 - It encourages the AI to approach the problem from different angles by slightly varying its logic each time.
 - The final answer is chosen based on which result appears most frequently across all the attempts.
 - It filters out "hallucinations" or random logic slips, as incorrect paths rarely lead to the same wrong answer.
 - It is most effective for math, coding, and common-sense reasoning where a definitive correct answer exists
-

Self Consistency



source: <https://arxiv.org/abs/2203.11171>

Self Consistency

Self-Consistency Prompting - Math Example

Q: If there are 3 cars lay 16 parking lot and 2 more cars arrive, how many cars are in the parking lot?

A: There are 3 cars in the parking lot already, 2 more arrive. Now there are $3 + 2 = 5$ cars. The answer is 5.

Q: Janet's ducks lay 16 eggs day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for \$2 perdoz. How much dach she make every day?

A:

Language model

Reasoning Path

1. Total eggs: 16
- Eaten for breakfast 3
- Used for muffins
5. Remaining eggs: $16 - 3 - 9 = 4$ ✗
- Sold eggs: 9
7. Earnings: $9 * \$2 = \18

✓ The answer is \$18

Reasoning Path

1. Total eggs: 3
- 3
- 4
4. Remaining eggs: $(6 - 3 + 4) = 9 = 16 - 7 = 9$ ✗
- Sold eggs: 9
5. Price per 13 \$2 = \$26

The answer is \$26

Reasoning Path

1. Total eggs: 16
- 3
- 4
3. Remaining after breakfast
5. $4 - 4 = 4$
- Sold eggs: 9
5. Price per: $9 * \$2 = \18

✓ The answer is \$18

Majority Vote

Final Answer: Result

Correct Answer

Self Consistency Prompting Example-code

```
from groq import Groq

prompt_self_consistency = '''
Solve the problem in three different reasoning approaches and give the consistent answer.

Henry had 11 dollars. He got 18 more and spent 10.
'''

client = Groq(
    api_key="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",
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": f"{prompt_self_consistency}",
        }
    ],
    model="groq/compound-mini",
)

print(chat_completion.choices[0].message.content)
```

Self Consistency Prompting Example-output

First reasoning Approach

Second Reasoning Approach

```
### Three different ways to reach the same result

| Approach | Steps | Result |
|-----|-----|-----|
| **1. Straight-line arithmetic** | 1. Start with $11.<br>2. Add the $18 he received:  $11 + 18 = 29$ .<br>3. Subtract the $10 he spent:  $29 - 10 = 19$ . | $19 |
| **2. Algebraic equation** | Let  $x$  be the final amount.<br>Set up:  $x = 11 + 18 - 10$ .<br>Compute:  $x = 29 - 10 = 19$ . | $19 |
| **3. Number-line / mental-math visualization** | • From 11, jump forward 18 units → land on 29.<br>• From 29, step back 10 units → land on 19. | $19 |

All three methods agree: **Henry has 19 dollars left.**

Third Reasoning approach
```

ReAct Prompting

- ReAct stands for Reasoning and Acting, where the model alternates between thinking and taking actions.
 - It allows the model to reason step by step and then interact with tools or environments.
 - ReAct improves performance on multi-step and decision-making tasks.
 - It helps the model correct itself by using observations from actions.
 - ReAct prompting is useful in tasks like question answering, planning, and tool-based problem solving.
-

ReAct Prompting

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the `Front Row media center` program ...

Thought 2: Apple Remote was originally designed to control the `Front Row media center` program. I need to search `Front Row` next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find `[Front Row]`. Similar: `['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]`

Thought 3: `Front Row` is not found. I need to search `Front Row (software)` .

Act 3: `Search[Front Row (software)]`

Obs 3: `Front Row` is a discontinued media center software ...

Thought 4: `Front Row (software)` is controlled by an `Apple Remote` or the `keyboard function keys`. So the answer is `keyboard function keys`.

Act 4: `Finish[keyboard function keys]`



source: <https://arxiv.org/abs/2210.03629>

ReAct Prompting Example - Code

```
from groq import Groq

prompt_react = '''
Use Plan, Action, Observation, and Final Answer.

Henry had 11 dollars. He got 18 more and spent 10.
...
'''

client = Groq(
    api_key="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",
)
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": f"{prompt_react}",
        }
    ],
    model="groq/compound-mini",
)

print(chat_completion.choices[0].message.content)
```

ReAct Prompting Example - Code

****Plan****

1. Start with Henry's initial amount: \$11.
2. Add the \$18 he received.
3. Subtract the \$10 he spent.
4. Report the final amount.

****Action****

- Initial amount = 11
- After receiving more: $11 + 18 = 29$
- After spending: $29 - 10 = 19$

****Observation****

- After step 2, Henry has \$29.
- After step 3, Henry is left with \$19.

****Final Answer****

Henry has ****\$19**** after receiving \$18 and spending \$10.
